ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Эмулятор Siemens/Infineon C166S v1 classic для динамического reverse engineering

Инструмент для динамического реверс-инжиниринга программ, написанных для Siemens/Infineon C166 classic

Версия	1.2
Статус	Рабочая версия
Дата	2026-10-22
Ментор	Кутаев Кирилл
Организация	MIPT x PT

Содержание

1 Общие сведения ... 3
1.1 Назначение документа ... 3
1.2 Термины и определения ... 3
2 Задачи и цели ... 4
2.1 Цель ... 4
2.2 Задачи ... 4
3 Архитектура ... 5
3.1 Общее представление ... 5
3.2 Описание всех компонентов ... 5
3.2.1 1. Registers ... 5
3.2.2 2. Program Status Word ... 5
3.2.3 3. Arithmetic Logic Unit (ALU) ... 6
3.2.4 4. BitProcessor ... 6
3.2.5 5. Stack + BranchUnit ... 7
3.2.6 6. Memory ... 8
3.2.7 7. MoveUnit ... 8
4 Ограничения ... 9
4.1 Обязательно ... 9
4.2 Желательно ... 9
5 Технологический стек ... 10
6 Календарный план ... 11
6.1 Бош Дмитрий (блок Memory) ... 11
6.2 Субботин Александр (блок Fetcher) и Мелешенко Александр (блок Executor) ... 12
6.3 Сочкова Надежда ... 13

Общие сведения

Назначение документа

Настоящее техническое задание определяет:

- поддерживаемую модель процессора C166 classic;
- состав реализуемых инструкций;
- модель регистров и PSW;
- модель памяти и адресации;
- порядок выполнения инструкции;
- правила обработки ошибок, traps и прерываний;
- требования к тестированию и результатам версии v1.

Термины и определения

Термин	Определение
ISA	Сокращение наименования Siemens/Infineon C166 classic
BYTE	Один байт памяти
WORD	Два байта памяти
DWORD	Четыре байта

Задачи и цели

Цель

Цель проекта — разработать программный эмулятор процессорного ядра Siemens/Infineon C166 classic, предназначенный для динамического анализа и исследования исполняемого кода.

В рамках первой версии реализуется эмулятор, умеющий работать с регистрами, арифметическими и логическими операциями, адресацией памяти, управлением потоком исполнения и операциями со стеком.

Задачи

1. **Исследовать архитектуру ISA**

   Изучить документацию на ISA. Требуется определить состав регистров, флагов и изучить работу с памятью. Установить перечень поддерживаемых инструкций. Отдельного внимания заслуживает работа с 24-битной физической памятью при помощи DPP.

2. **Реализация компонентов**

   Реализация основных компонентов эмулятора. Про каждый из них написано подробнее в разделе **Архитектура**.

3. **Тестирование**

   Каждый компонент должен быть протестирован отдельно. Также нужно протестировать работу эмулятора в целом, используя готовые бинарные файлы с известным поведением.

4. **Документирование**

   Написание документации по проекту и инструкций по использованию эмулятора.

Архитектура

Общее представление

Цикл ядра представляет собой следующую цепочку событий:

$$\text{Fetch} \rightarrow \text{Decode} \rightarrow \text{Execute} \rightarrow \text{WriteBack}$$

Во время выполнения цикла эти четыре компонента могут обращаться к другим, например:

- Registers
- PSW
- Memory
- ALU
- BitProcessor
- Stack
- BranchUnit
- MoveUnit

(Взаимодействие между компонентами пока показано на черновой схеме `arch.png`.)

Описание всех компонентов

1. Registers

Содержит в себе все регистры процессора и предоставляет интерфейсы для взаимодействия с ними:

Регистры

R0 - R15	Регистры общего назначения
Stack Pointer	Указатель на стек
Code Segment Pointer	Указатель на начало сегмента кода
Instruction Pointer	Адрес инструкции в текущем сегменте кода
Data Page Pointer 0 - 3	Указатель на страницу памяти
Context Pointer	Адресация текущего контекста
Program Status Word	Флаги состояния процессора
MDH и MDL (вместе MD)	Регистры для умножения и деления (32 бита)

Для регистров R0-R7 есть возможность адресовываться отдельно к старшему и младшему байту (RH0-RH7 - старший байт; RL0-RL7 - младший байт). Компонент должен предоставлять интерфейс для чтения и записи значений регистров.

2. Program Status Word

Содержит в себе флаги процессора. Компонент предоставляет доступ к чтению / записи любого флага из этого регистра.

Флаги

N (negative)	Флаг знака (1 - если результат ALU отрицателен)
C (carry)	Флаг беззнакового переполнения
V (overflow)	Флаг знакового переполнения
Z (zero)	Флаг нуля (1 - если результат ALU равен 0)
USR0	Пользовательский флаг

3. Arithmetic Logic Unit (ALU)

Компонент для арифметики. Должен уметь выполнять базовые арифметические операции, возвращать результат и изменение флагов.

Операции ALU

ADD	Сложение
ADDB	Сложение (байтовое)
ADDC	Сложение с учетом флага C от предыдущей операции
ADDCB	Сложение с учетом флага C от предыдущей операции (байтовое)
SUB	Вычитание
SUBB	Вычитание (байтовое)
SUBC	Вычитание с учетом флага C от предыдущей операции
SUBCB	Вычитание с учетом флага C от предыдущей операции (байтовое)
MUL	Умножение
MULU	Беззнаковое умножение
DIV	Деление
DIVL	Деление (32 бит)
DIVU	Беззнаковое деление
DIVLU	Беззнаковое деление (32 бит)
CMP	Сравнение операндов (изменяет только флаги)
CMPB	Сравнение операндов (байтовое) (изменяет только флаги)
NEG	Арифметическое отрицание

4. BitProcessor

Компонент, аналогичный ALU, который предоставляет операции с битами.

Операции BitProcessor

AND	Побитовое И
OR	Побитовое ИЛИ
XOR	Побитовое исключающее ИЛИ
CPL	Побитовое НЕ
SHL	Сдвиг влево
SHR	Сдвиг вправо
ASHR	Арифметический сдвиг вправо (сохраняется знаковый бит)
ROL	Циклический сдвиг влево
ROR	Циклический сдвиг вправо

5. Stack + BranchUnit

Компонент, эмулирующий работу стека. Содержит в себе Stack Pointer (SP), механизмы контроля переполнения и опустошения стека (STKOV и STKUN), интерфейсы для реализации вызова функции (CALL) и возврата (RET). Ветвление реализуется с помощью условных вызовов подпрограмм.

Операции Stack

PUSH	Уменьшает SP на 2 и записывает WORD на стек
POP	Читает WORD со стека и увеличивает SP на 2
CALL	Сохраняет адрес возврата на стеке и передает управление функции
RET	Читает адрес возврата и передает управление по нему
RETS	Читает адрес возврата и сегмент и передает управление по ним
RETP	Читает адрес возврата и сохраненный регистр и передает управление по нему, восстанавливая значение регистра
STKOV	Регистр для контроля переполнения стека
STKUN	Регистр для контроля пустоты стека
CALLA	Если выполняется условие, передать управление функции (абсолютный адрес)
CALLI	Если выполняется условие, передать управление функции (адрес функции берется из регистра)
CALLR	Безусловно передать управление функции (относительный адрес)
CALLS	Безусловно передать управление функции из другого сегмента
PCALL	Передать управление функции, сохранив определенный регистр на стеке (восстановится после оператора RETP)

JMPA	Условный переход по абсолютному адресу
JMPI	Условный переход по адресу из регистра
JMPR	Условный переход по относительному адресу
JMPS	Безусловный переход по абсолютному адресу с учетом сегмента
JB	Условный переход, если установлен определённый бит
JNB	Условный переход, если НЕ установлен определённый бит
JBC	Условный переход, если установлен определённый бит (бит сбрасывается)
JNBS	Условный переход, если НЕ установлен определённый бит (бит устанавливается)

6. Memory

Этот компонент должен предоставлять интерфейс для чтения и записи данных и учитывать механизмы DPP.

Операции Memory

read8	Прочитать BYTE по адресу
read16	Прочитать WORD по адресу
write8	Записать BYTE по адресу
write16	Записать WORD по адресу
LogicToPhysic	Перевести 16-битный логический адрес в 24-битный физический, учитывая механизм DPP

7. MoveUnit

Компонент, который предоставляет интерфейс для всех вариаций операции MOV (копирование из одного места в другое).

Операции MoveUnit

MOV	Скопировать значение правого операнда в левый (Оба операнда WORD)
MOVB	Скопировать значение правого операнда в левый (Оба операнда BYTE)
MOVBS	Скопировать значение правого операнда в левый (Из BYTE в WORD с сохранением бита знака)
MOVBZ	Скопировать значение правого операнда в левый (Из BYTE в WORD, старший байт результата зануляется)

Ограничения

В силу ограниченности по времени предлагается выделить два блока: обязательное и желательное. Проект считается завершенным, если реализовано все из блока *обязательно*.

Обязательно

1. Загрузка бинарного файла в память.
2. Установка начального состояния процессора.
3. Запуск исполнения.
4. Пошаговое исполнение одной инструкции.
5. Просмотр текущих значений регистров и PSW.
6. Просмотр содержимого памяти.
7. Получение информации о текущей инструкции.
8. Остановка исполнения при достижении заданного адреса.
9. Остановка при неподдерживаемой инструкции или другой определённой ошибке исполнения.
10. Получение конечного состояния процессора после завершения тестовой программы.
11. Точки остановки.
12. Трассировка исполнения.
13. Журнал изменений регистров.
14. Журнал обращений к памяти.

Желательно

1. Сохранение и загрузка состояния эмулятора.
2. Экспорт трассировки для последующего анализа.
3. Step In / Step Out
4. Взаимодействие с IDA PRO

Технологический стек

C++ 23	Важна скорость исполнения, а также возможность впоследствии связать с IDA PRO.

Календарный план

Бош Дмитрий (блок Memory)

№	Этап / работа	Начало	Окончание
1	Изучение работы эмулятора, ознакомление с документацией C166S v1	28.09.26	11.10.26
2	Обсуждение архитектуры проекта и интерфейсов взаимодействия	12.10.26	25.10.26
3	Модуль физической адресации памяти	26.10.26	01.11.26
4	Интерфейс чтения и записи обычных данных с использованием механизма DPP	02.11.26	08.11.26
5	Чтение и запись регистров. Чтение и изменение указателя на следующую инструкцию PC (CSP:IP)	09.11.26	15.11.26
6	Reset памяти и загрузка бинарного файла в память. Журнал обращения к памяти и к регистрам	16.11.26	22.11.26
7	Тестирование и документирование	23.11.26	29.11.26
8	Подготовка к защите	30.11.26	07.12.26

Субботин Александр (блок Fetcher) и Мелешенко Александр (блок Executor)

№	Этап / работа	Начало	Окончание
1	Изучение работы эмулятора, ознакомление с документацией C166S v1	28.09.26	11.10.26
2	Обсуждение архитектуры проекта и интерфейсов взаимодействия	12.10.26	25.10.26
3	Блок ALU и MoveUnit	26.10.26	01.11.26
4	Инструкции по взаимодействию со стеком	02.11.26	08.11.26
5	Инструкции, связанные с ветвлением	09.11.26	15.11.26
6	Побитовые операции	16.11.26	22.11.26
7	Тестирование и документирование	23.11.26	29.11.26
8	Подготовка к защите	30.11.26	07.12.26

Сочкова Надежда

№	Этап / работа	Начало	Окончание
1	Изучение работы эмулятора, ознакомление с документацией C166S v1	28.09.26	11.10.26
2	Обсуждение архитектуры проекта и интерфейсов взаимодействия	12.10.26	25.10.26
3	CLI и реализация команд: Step, Run, AddBreakpoint, RemoveBreakpoint, ShowNextInstruction	26.10.26	01.11.26
4	Получение и вывод в консоль текущего состояния процессора	02.11.26	08.11.26
5	CLI для чтения и записи данных в память (по физическому адресу в обход DPP и по логическому адресу, учитывая DPP)	09.11.26	15.11.26
6	Обработка исключений и вывод их в терминал	16.11.26	22.11.26
7	Тестирование и документирование	23.11.26	29.11.26
8	Подготовка к защите	30.11.26	07.12.26